Bilan de notre séance

Gastronome Créatif | 10 octobre 2026

Voici le récapitulatif de notre travail sur la page de détail des recettes, pour reprendre facilement le projet demain.

1. Affichage de la note moyenne des recettes

Nous avons utilisé la propriété Moyenne du RecetteDetailViewModel, calculée à partir des avis, pour afficher une note moyenne sous forme d'étoiles.

```
int noteArrondie = (int)Math.Round(Model.Moyenne);
```

Une moyenne de 3,6/5 devient 4 après arrondi : quatre étoiles pleines et une étoile vide. Une boucle for affiche cinq symboles et une condition if / else choisit les étoiles pleines ou vides.

Au-dessus des avis, nous conservons aussi la moyenne numérique avec deux décimales, grâce à Model.Moyenne.ToString("F2").

2. Informations principales de la recette

Dans Detail.cshtml, nous avons travaillé sur le bloc .infos-principales qui affiche l'auteur, la note, le temps de préparation, la difficulté et un emplacement pour le prix.

Nous avons choisi de présenter la note uniquement en étoiles dans cette zone, afin de garder un affichage visuel et léger. La variable noteArrondie est déclarée avant son utilisation pour être accessible aux différentes sections de la vue.

3. Recettes sans avis

Une moyenne de zéro parce qu'il n'y a aucun avis ne doit pas être présentée comme une mauvaise note. Nous avons donc ajouté une condition qui affiche les étoiles uniquement lorsque des avis existent.

```
@if (Model.Avis.Count > 0)
{
    // Afficher les étoiles
}
else
{
    <span>Pas encore de note</span>
}
```

Nous avons également compris qu'une condition placée à l'extérieur d'une boucle est évaluée une seule fois, alors que si elle est placée à l'intérieur, elle est réévaluée à chaque itération.

4. Affichage dynamique de la difficulté

Le champ Model.Recette.difficulte contient une valeur de 1 à 5 (5 = difficulté maximale). Tu as réussi à écrire une boucle Razor de cinq itérations et une condition pour distinguer les symboles actifs des symboles inactifs.

```
@for (int i = 1; i <= 5; i++)
{
```

```
    if (i <= Model.Recette.difficulte)
    {
        <span class="etoile">★</span>
    }
    else
    {
        <span class="etoile-vide">☆</span>
    }
}
```

La difficulté étant déjà un entier, Math.Round() n'est pas nécessaire.

5. Différencier notes et difficulté

Nous avons choisi de ne pas utiliser les mêmes symboles pour deux informations différentes :

• Notes des utilisateurs : étoiles dorées.

• Difficulté : toques de chef, avec une couleur différente pour les niveaux actifs et inactifs.

Ce choix de design est acté, mais les toques ne sont pas encore intégrées dans le code.

6. Notions techniques pratiquées

Notion	Ce que nous avons pratiqué
Razor	Mélanger HTML et instructions C#
@for	Répéter cinq fois un affichage
if / else	Adapter le contenu à une condition
Math.Round()	Arrondir une moyenne
Portée des variables	Déclarer une variable réutilisable
Model.Avis.Count	Détecter la présence d'avis
	Afficher des éléments sur une même ligne
CSS	Centrer avec text-align: center
UX / UI	Distinguer deux informations visuellement

7. Point de reprise pour demain

• Intégrer une bibliothèque d'icônes adaptée dans _Layout.cshtml.

• Vérifier qu'une icône s'affiche dans Detail.cshtml.

• Adapter la boucle Razor pour afficher cinq toques selon la difficulté.

• Personnaliser les toques actives et inactives avec le CSS.

• Vérifier le résultat dans le navigateur.

Nous pourrons ensuite améliorer la présentation des ingrédients et poursuivre les fonctionnalités des boutons.

Bilan : tu as gagné en autonomie avec les boucles, les conditions, Razor et la réutilisation de logique dans ASP.NET Core MVC. Bon appétit et à demain !